

Homework 1 – Part 2: Implementation & Visualization (MuJoCo/ROS)

Name: Anjil Shah (Ph.D. Mechanical)

Roll No: 26310010

GitHub link: <https://github.com/anjilshah/ME639-HW1-Part2-AnjilShah>.

❖ Problem 7 – Rotation Sandbox: Does Order Matter?

- **Objective**

The objective of Problem 7 was to use the provided asymmetric-body MuJoCo model to apply elemental rotations about the x , y , and z axes. For each rotation, the reference frame can be either the current body frame or the fixed space frame. The experiment was used to reproduce visually the non-commutativity of rotations. An asymmetric body was used so that differences in the final orientation could be easily observed.

- **Rotation Sequence**

For the main experiment, I used the following two rotations:
 $Z(+90^\circ)$ followed by $X(+90^\circ)$.

Two cases were compared.

- **Experiment 1 – Current/Body Frame**

The first rotation was performed about the current body Z_b axis and the second rotation was performed about the new current body X_b axis.
 $Z_b(+90^\circ) \rightarrow X_b(+90^\circ)$

- **Experiment 2 – Fixed/Ground Frame**

The same angles and axes were used, but the rotations were taken about the fixed space-frame axes.
 $Z_g(+90^\circ) \rightarrow X_g(+90^\circ)$ where X_b, Y_b, Z_b represent the body frame and X_g, Y_g, Z_g represent the fixed ground/space frame.

- **Rotation Matrices**

For Experiment 1,

$$R_1 = R_z(90^\circ)R_x(90^\circ).$$

This gives

$$R_1 = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

For Experiment 2,

$$R_2 = R_x(90^\circ)R_z(90^\circ).$$

This gives

$$R_2 = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Therefore, $R_1 \neq R_2$

This difference in rotation matrices results in different final orientations of the asymmetric body.

- **MuJoCo Visualization**

The body was visualized using the asymmetric MuJoCo model. The body-frame axes were represented using:

X_b, Y_b, Z_b and the fixed ground-frame axes using:

X_g, Y_g, Z_g . The body axes rotate together with the asymmetric object, while the ground-frame axes remain fixed.

The final interface also displayed the rotation matrices and the comparison

- **R1**

to make the mathematical connection with the visual result clear.

- **Result and Discussion**

The experiment showed that the same two elemental rotations do not necessarily produce the same final orientation when they are taken about different reference frames. The asymmetric shape of the body made the difference visually clear.

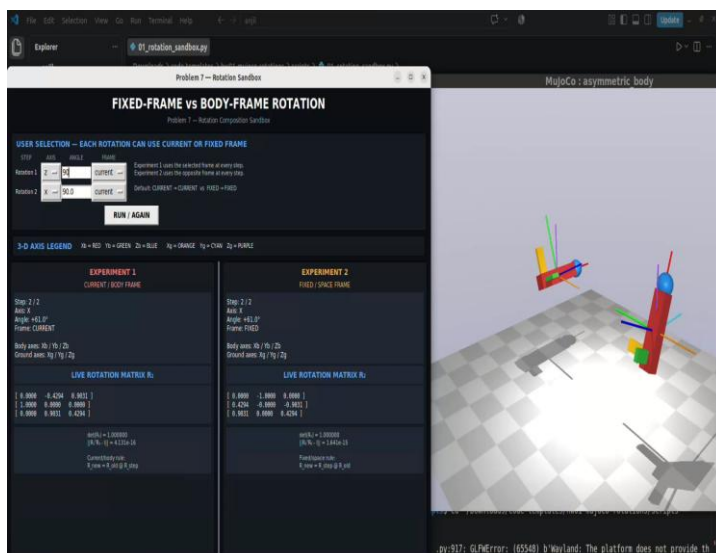
This demonstrates the non-commutative nature of three-dimensional rotations and connects the simulation result to the rotation-composition concepts studied earlier in the homework. The link for the screen recording is provided below.

Outcome of the problem:

The screen recording is present here for this problem:

<https://youtu.be/JtrZr5Mzqf0>.

Below are the images Fig 7.1 and 7.2 that show the development visualization of the two rotation experiments, showing the current/body frame and fixed/space frame and two rotation matrices showing $R1 \neq R2$ respectively.



AI use Note Problem:7

Tool used: ChatGPT

What I asked it:

I got a Not Implemented Error in 01_rotation_sandbox.py at compose_sequence(). I asked how to implement current-frame versus fixed-frame rotation composition correctly.

How I used the output (and what I changed/verified myself):

I used the explanation to understand the right- and left-multiplication rules, then edited the code myself in VS Code and ran the MuJoCo simulation to verify that the two cases produced different final orientations.

❖ Problem 8 – Verifying the Skew-Symmetric Identities in Simulation.

- **Objective:**

The objective of Problem 8 was to numerically verify the two identities established in Problem 5 using an actual MuJoCo simulation.

The body was simulated under a time-varying angular velocity, and the actual rotation matrix $R(t)$ generated by MuJoCo was used for the numerical checks.

- **Identities Tested**

The first identity was

$R(v \times w) = (Rv) \times (Rw)$ The second identity was

$$R\hat{\omega}R^T = \hat{R\omega}$$

The `hat()` operator supplied in `utils.py` was used to construct the skew-symmetric matrix of ω .

- **Simulation Procedure**

A time-varying angular velocity was applied to the simulated body.

The simulation was evaluated at five logged time points. At every logged time point, five random vectors $v, w, \omega \in R^3$ were generated.

For the first identity, the residual was calculated as

$$R(v \times w) - (Rv) \times (Rw)$$

For the second identity, the residual was calculated as

$$\|R\hat{\omega}R^T - \hat{R\omega}\|.$$

The maximum residual at each time point was recorded.

- **Numerical Results**

The numerical results obtained from the simulation were:

Identity	Maximum Residual	Mean Residual
$R(v \times w) - (Rv) \times (Rw)$	1.289×10^{-15}	8.201×10^{-16}
$R\hat{\omega}R^T - \hat{R\omega}$	1.156×10^{-15}	7.390×10^{-16}

The residuals are extremely small and are at approximately floating-point machine precision.

The values of approximately 10^{-15} are consistent with the expected round-off error of floating-point calculations.

```
Warning: NaN message, var=2.7446,
Step 1: t=1.0000s, |omega|=2.7446, cross=5.088e-16, skew=6.336e-16
Step 2: t=2.0000s, |omega|=2.6870, cross=9.155e-16, skew=1.156e-15
Step 3: t=3.0000s, |omega|=2.8288, cross=6.844e-16, skew=9.213e-16
Step 4: t=4.0000s, |omega|=2.7140, cross=1.289e-15, skew=6.465e-16
Step 5: t=5.0000s, |omega|=2.1705, cross=7.022e-16, skew=3.380e-16

=====
PROBLEM 8 – FINAL NUMERICAL RESULTS
=====

Identity 1:
R(v x w) = (Rv) x (Rw)
Maximum residual = 1.289e-15
Mean residual = 8.201e-16

Identity 2:
R hat(w) R^T = hat(Rw)
Maximum residual = 1.156e-15
Mean residual = 7.390e-16

CSV saved to:
/home/anjil/Downloads/code-templates/hw01-mujoco-rotations/scripts/problem8_residuals.csv
```

Above is the image Fig 8.1 showing the final panel of the problem 8.

Here is the link of csv file for the numerical result:

<https://drive.google.com/file/d/1OYHun3hAgnxb5r06Hdie2fAbr6YOBCAI/view?usp=sharing>.

- **Conclusion for Problem 8**

Both identities produced residuals close to 10^{-15} . Therefore, the numerical experiment strongly supports the analytical results obtained in Problem 5.

- **AI use Note Problem:8**

Tool used: ChatGPT

What I asked it:

I asked for help implementing the two residual checks and debugging an error while saving the numerical results to CSV.

How I used the output (and what I changed/verified myself):

I implemented the numerical checks for both identities, added the time-varying angular velocity and logged simulation results, and fixed the CSV field-name error. I reran the simulation myself and verified residuals of approximately.

Problem 9 – Current Frame vs. Fixed Frame in ROS/RViz

- **Objective**

The objective of Problem 9 was to extend the current-frame versus fixed-frame rotation experiment into ROS 2 and RViz2.

The body frame and space frame were represented using ROS TF. A ROS parameter was used to switch between current-frame and fixed-frame composition while RViz2 was running.

- **ROS 2 Package**

The ROS 2 package used for the experiment was:

hw01_tf_demo

The main executable was:

tf_broadcaster_node

The TF frames were organized as:

```
world
├── space_frame
└── body_frame
```

The space_frame represents the fixed reference frame, while body_frame represents the rotating body.

- **Live Frame Toggle**

The composition mode was controlled using the ROS parameter:

compose_frame

The parameter accepts:

current or fixed

For example:

```
ros2 param set /hw01_tf_broadcaster compose_frame fixed
```

and:

```
ros2 param set /hw01_tf_broadcaster compose_frame current
```

The parameter was changed while the ROS node and RViz2 were running. Changing the parameter resets the rotation sequence so that the effect of the selected frame convention can be observed directly.

- **RViz2 Visualization**

RViz2 was used to visualize the TF frames.

The fixed frame was set to:

`space_frame`

and a TF display was added.

The intended TF structure is:

world

├── `space_frame`

└── `body_frame`

The `space_frame` remains fixed while the orientation of `body_frame` changes according to the selected composition convention.

- **Verification**

The ROS node was successfully started using:

```
ros2 run hw01_tf_demo tf_broadcaster_node
```

The available ROS parameters included:

```
compose_frame  
step_period  
use_sim_time
```

The `compose_frame` parameter was successfully changed during execution between `current` and `fixed`.

The `/tf` topic was also checked and the fixed `space_frame` transform was observed.

Additional RViz2 verification was used to observe the body-frame transformation.

Outcomes:

```
[INFO] [1788247468.012726575] [hw01_tf_broadcaster]: [CURRENT] Step 1: Z +90.0 degrees
[INFO] [1788247468.014027314] [hw01_tf_broadcaster]: R_body =
[[ 1.  0.  0.]
 [-0. -0.  1.]
 [ 0. -1. -0.]]
[INFO] [1788247469.012908901] [hw01_tf_broadcaster]: [CURRENT] Step 2: X +90.0 degrees
[INFO] [1788247469.014420619] [hw01_tf_broadcaster]: R_body =
[[ 1.  0. -0.]
 [-0.  1.  0.]
 [ 0. -0.  1.]]
[INFO] [1788247470.013116463] [hw01_tf_broadcaster]: [CURRENT] Step 1: Z +90.0 degrees
[INFO] [1788247470.014552897] [hw01_tf_broadcaster]: R_body =
[[ 0. -1. -0.]
 [ 1.  0.  0.]
 [-0. -0.  1.]]
[INFO] [1788247471.012686539] [hw01_tf_broadcaster]: [CURRENT] Step 2: X +90.0 degrees
[INFO] [1788247471.014050018] [hw01_tf_broadcaster]: R_body =
[[ 0. -0.  1.]
 [ 1.  0. -0.]
 [-0.  1.  0.]]

[INFO] [1788247514.012106806] [hw01_tf_broadcaster]: [CURRENT] Step 1: Z +90.0 degrees
[INFO] [1788247514.012788947] [hw01_tf_broadcaster]: R_body =
[[ -0. -0.  1.]
 [ 0. -1. -0.]
 [ 1.  0.  0.]]
[INFO] [1788247514.649666606] [hw01_tf_broadcaster]: -----
[INFO] [1788247514.650393988] [hw01_tf_broadcaster]: compose frame changed to: fixed
[INFO] [1788247514.651078999] [hw01_tf_broadcaster]: Using fixed/space-frame composition:
[INFO] [1788247514.651898133] [hw01_tf_broadcaster]: R_new = R_step @ R_old
[INFO] [1788247514.652708492] [hw01_tf_broadcaster]: Rotation sequence Reset.
[INFO] [1788247514.653923813] [hw01_tf_broadcaster]: -----
[INFO] [1788247515.012914676] [hw01_tf_broadcaster]: [FIXED] Step 1: Z +90.0 degrees
[INFO] [1788247515.014253949] [hw01_tf_broadcaster]: R_body =
[[ 0. -1.  0.]
 [ 1.  0.  0.]
 [ 0.  0.  1.]]
[INFO] [1788247516.011887707] [hw01_tf_broadcaster]: [FIXED] Step 2: X +90.0 degrees
[INFO] [1788247516.012672767] [hw01_tf_broadcaster]: R_body =
[[ 0. -1.  0.]
 [ 0.  0. -1.]
 [ 1.  0.  0.]]
[INFO] [1788247517.013002296] [hw01_tf_broadcaster]: [FIXED] Step 1: Z +90.0 degrees
[INFO] [1788247517.014305350] [hw01_tf_broadcaster]: R_body =
[[ -0. -0.  1.]
 [ 0. -1. -0.]
 [ 1.  0.  0.]]
[INFO] [1788247518.012956805] [hw01_tf_broadcaster]: [FIXED] Step 2: X +90.0 degrees
[INFO] [1788247518.014273572] [hw01_tf_broadcaster]: R_body =
```

Fig. 9.1. Live switching from current-frame to fixed-frame composition using the ROS `compose_frame` parameter. Left image shows current frame and Right shows fixed frame.

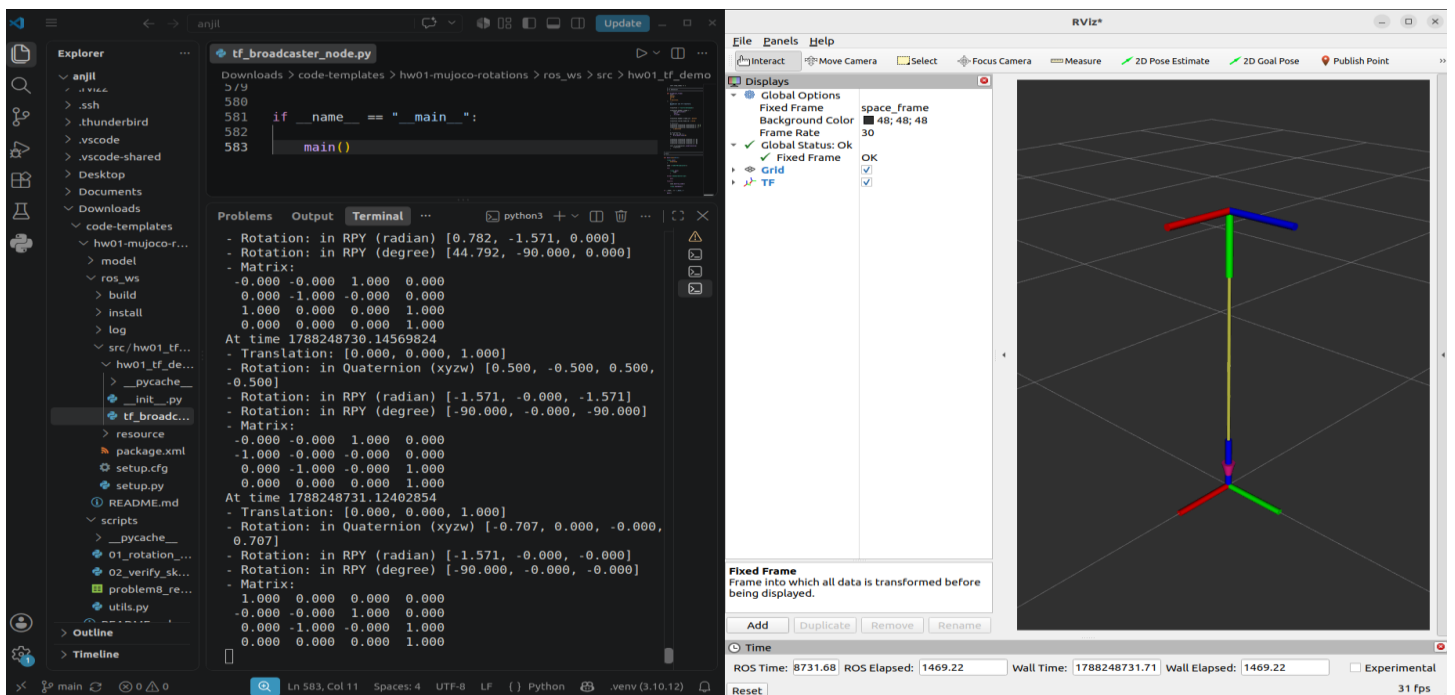


Fig.9.2 ROS 2 and RViz2 verification of the TF rotation experiment, showing the `tf_broadcaster_node.py` implementation, the live rotation-matrix output in the terminal, and the `space_frame` reference with the rotating `body_frame` visualized in Rviz2.

- **Conclusion for Problem 9**

The ROS 2 implementation provides a live representation of the difference between current-frame and fixed-frame rotation composition. The use of TF makes the reference frames explicit, while the `compose_frame` parameter allows the composition convention to be changed during execution.

- **AI use Note Problem:9**

Tool used:

ChatGPT

What I asked it:

I was unsure how to set up the ROS 2 TF frames and how to use the `compose_frame` parameter while RViz2 was running. I asked for help understanding the ROS 2 package structure and the live parameter toggle.

How I used the output (and what I changed/verified myself):

I used the guidance to work through the ROS 2 setup in VS Code. I built the package, ran the TF broadcaster, checked the available parameters, changed `compose_frame` between `current` and `fixed`, and tested the TF output in RViz2 myself.